



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaia School of Engineering
(formerly K J Somaia College of Engineering)



Combinational Circuits

Digital Logic Design

FY. BTech





SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Introduction

- Logic circuits for digital systems may be combinational or sequential.
- A logic circuit is combinational if its outputs at any time are a function of only the present inputs.
- A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.
- In contrast, sequential circuits employ storage elements in addition to logic gates.
- In sequential circuits outputs are a function of the inputs and the state of the storage elements. Because the state of the storage elements is a function of previous inputs to the circuit, the outputs of a sequential circuit depend at any time on not only the present values of inputs but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.

Combinational Circuit

- A combinational circuit consists of an interconnection of logic gates.
- Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data.
- A block diagram of a combinational circuit is shown in Figure below.
- The n input binary variables come from an external source; the m output variables are produced by the input signals acting on the internal combinational logic circuit, and go to an external destination.
- Each input and output variable exists physically as an analog signal¹ whose values are interpreted to be a binary signal that represents logic 0 and logic 1.

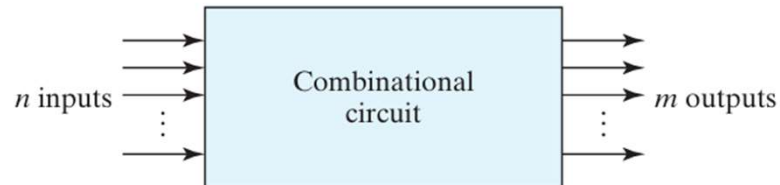


FIGURE 4.1

Block diagram of combinational circuit



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



COMPARISON

Aspect	Combinational Circuit	Sequential Circuit
Definition	Output depends only on the current inputs.	Output depends on both current inputs and past states (memory).
Memory Elements	Does not require memory elements.	Requires memory elements like flip-flops or latches.
Timing Dependency	Output is immediate, based on input changes.	Output is dependent on clock pulses and previous states.
Clock Signal	No clock signal required.	Requires a clock signal to synchronize state changes.
Design Complexity	Simpler design without the need for memory.	More complex due to memory and clock management.
Speed	Faster, as outputs change instantly with inputs.	Slower due to dependency on clock cycles and past states.
Functionality	Performs basic logical operations without sequence dependency.	Performs operations that require sequences or timed events.
Examples	Adders, Subtractors, Multiplexers, Encoders.	Counters, Shift Registers, Flip-Flops, State Machines.
Power Consumption	Generally lower power consumption.	Higher power consumption due to memory and clock circuitry.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaia School of Engineering
(formerly K J Somaia College of Engineering)



Binary Adder-Subtractor

- Digital computers perform a variety of information-processing tasks. Among the functions encountered are the various arithmetic operations.
- The most basic arithmetic operation is the addition of two binary digits.
- A combinational circuit that performs the addition of two bits is called a half adder.
- One that performs the addition of three bits (two significant bits and a previous carry) is a full-adder.
- The names of the circuits stem from the fact that two half adders can be employed to implement a full adder.
- A binary adder–subtractor is a combinational circuit that performs the arithmetic operations of addition and subtraction with binary numbers.
- The subtraction circuit is included in a complementing circuit.

HALF ADDER

- A half adder circuit has two binary inputs and two binary outputs.
- A combinational circuit that performs the addition of two bits is called a half adder.
- The input is considered as 'x' and 'y' the circuit has two outputs sum 'S' and carry 'C'
- The truth table is shown below:

Table 4.3
Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

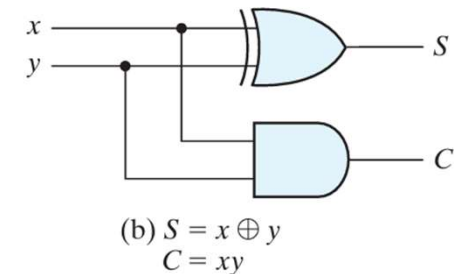
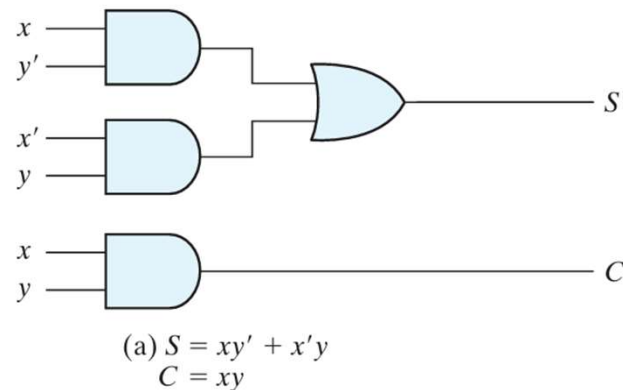


FIGURE 4.5
Implementation of half adder

HALF SUBTRACTOR

- A logic circuit for the subtraction of **B** (subtrahend) from **A** (minuend) where **A** and **B** are 1-bit numbers is referred to as a half-subtractor.
- The subtraction process is illustrated below and is reproduced in truth table form.
- Here, A and B are the two inputs and **D** (difference) and **C** (borrow) are the two outputs.

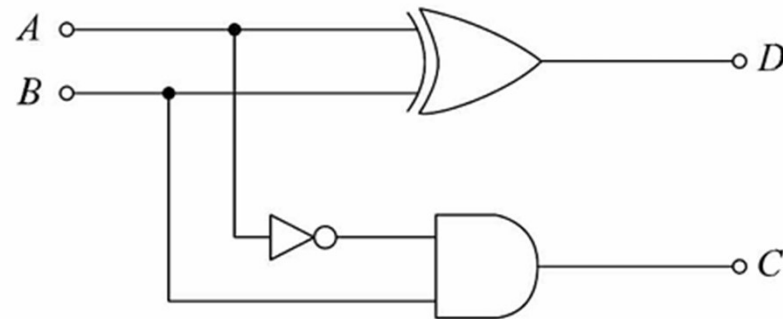


Fig. 5.22 *Realisation of a Half-subtractor*

HALF SUBTRACTOR

Table 5.13 *Truth Table of a Half-subtractor*

Inputs		Outputs	
<i>A</i>	<i>B</i>	<i>D</i>	<i>C</i>
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

From the truth table, the logical expressions for *D* and *C* are obtained as

$$D = \bar{A}B + A\bar{B} = A \oplus B \quad (5.36a)$$

$$C = \bar{A}B \quad (5.36b)$$

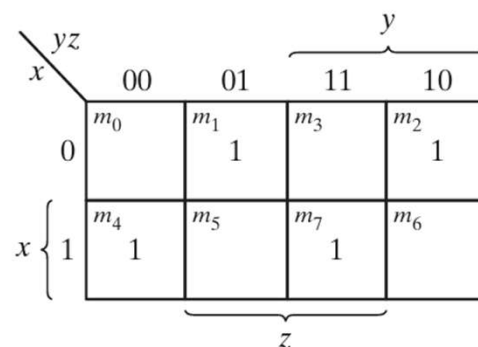
FULL ADDER

- A full adder is a combinational circuit that forms the arithmetic sum of three bits.
- It consists of three inputs and two outputs.
- Two of **the input variables**, denoted by '**x**' and '**y**', represent the two significant bits to be added. The third input, '**z**', represents **the carry from the previous lower significant position**.
- Two outputs are necessary because the arithmetic sum of three binary digits ranges in decimal value from 0 to 3, and binary representation of the decimal digits 2 or 3 needs two bits.
- The two outputs are designated by the symbols '**S**' for sum and '**C**' for carry.
- The binary variable '**S**' gives the value of the **least significant bit of the sum**.
- The binary variable '**C**' gives the **output carry** formed by adding the input carry and the bits of the words.

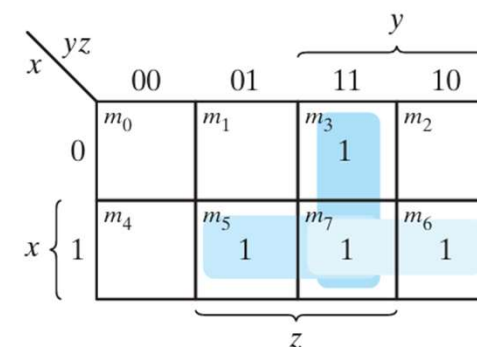
FULL ADDER (cont.)

Table 4.4
Full Adder

<i>x</i>	<i>y</i>	<i>z</i>	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



(a) $S = x'y'z + x'yz' + xy'z' + xyz$



(b) $C = xy + xz + yz$

FIGURE 4.6
K-Maps for full adder

FULL ADDER (cont..)

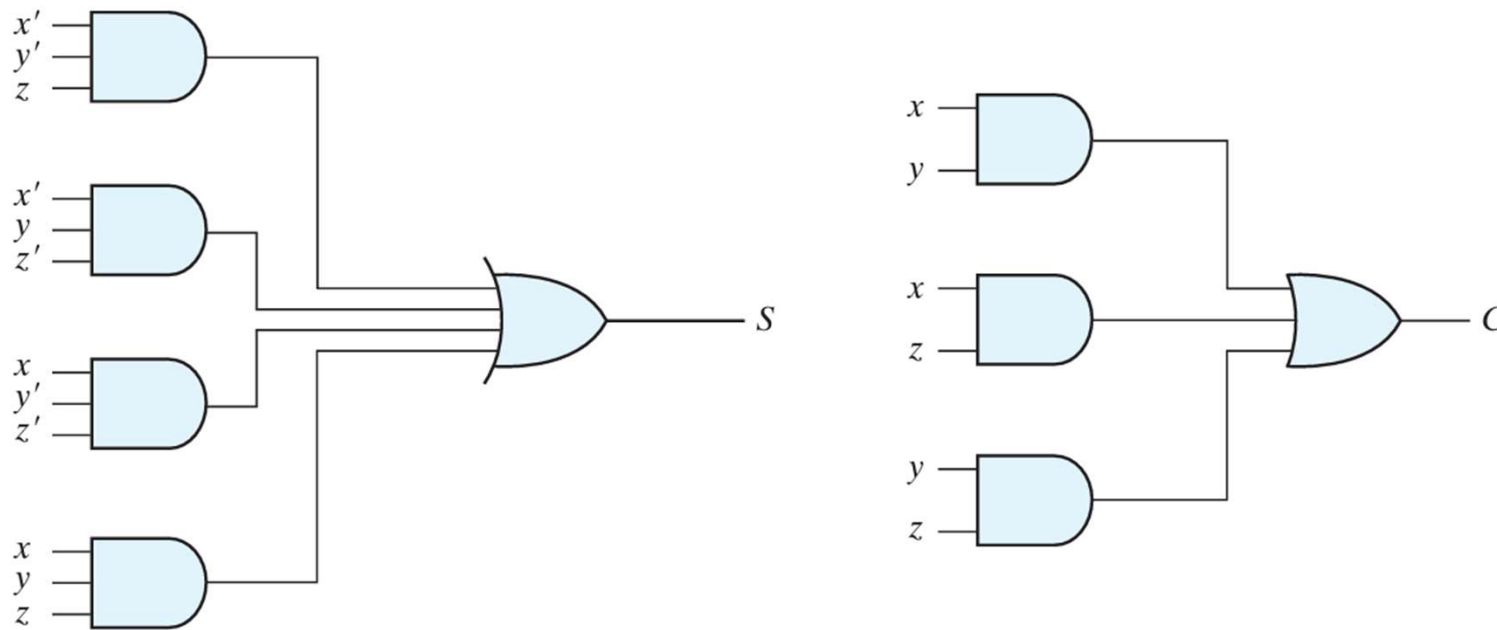


FIGURE 4.7

Implementation of full adder in sum-of-products form



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



FULL SUBTRACTOR

- Just like a full-adder, a full-subtractor circuit is required for performing multibit subtraction wherein a borrow from the previous bit position may also be present.
- A full-subtractor features three inputs: A_n (minuend), B_n (subtrahend), and C_{n-1} (borrow from the previous stage).
- The circuit produces two outputs: D_n (difference) and C_n (borrow).
- The K-map for the output D_n is identical to the K-map for S_n of the adder circuit.

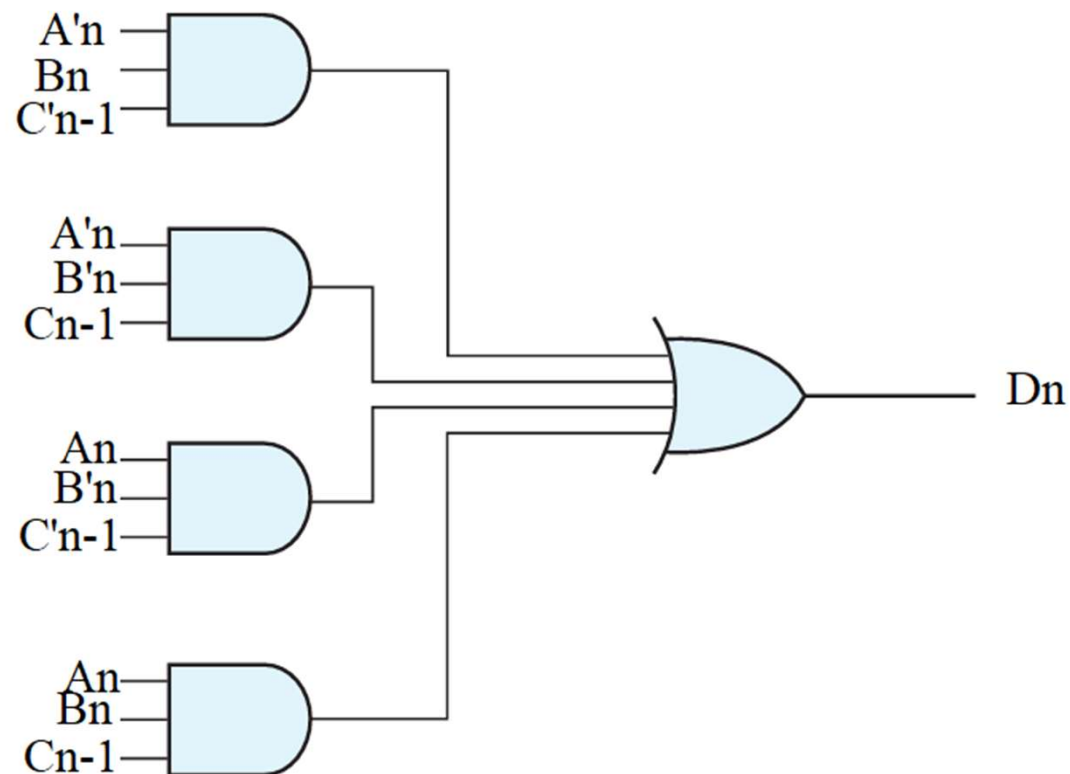
FULL SUBTRACTOR (cont.)

		$A_n B_n$			
		00	01	11	10
C_{n-1}	0		1		1
	1	1		1	

K-map for D_n

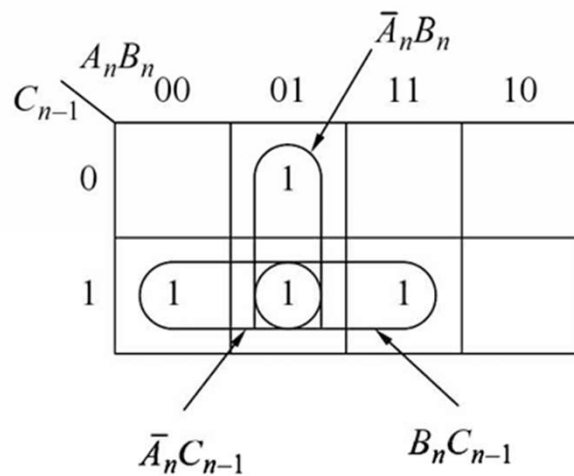
(a)

$$D_n = \bar{A}_n B_n \bar{C}_{n-1} + \bar{A}_n \bar{B}_n C_{n-1} + A_n \bar{B}_n \bar{C}_{n-1} + A_n B_n C_{n-1}$$

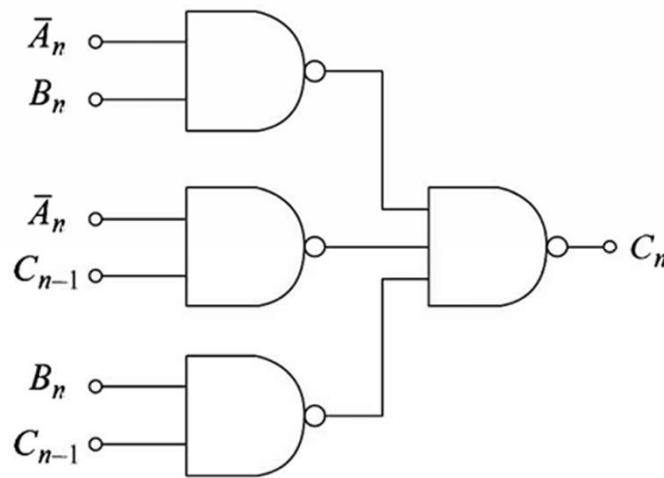


FULL SUBTRACTOR (cont..)

$$C_n = \bar{A}_n B_n + \bar{A}_n C_{n-1} + B_n C_{n-1}$$



(a)



(b)

Fig. 5.23 (a) K-map for C_n (b) Realisation of C_n

FULL ADDER USING TWO HALF ADDERS

Implement Full Adder using two half adder & a basic gates.

Solution:-

Truth table for Full Adder

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned} \text{Sum} &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + AB C_{in} \\ &= C_{in}(\bar{A}\bar{B} + AB) + \bar{C}_{in}(\bar{A}B + A\bar{B}) \\ &= C_{in}(A \oplus B) + \bar{C}_{in}(A \oplus B) \\ &= A \oplus B \oplus C \end{aligned}$$

$$\begin{aligned} \text{Carry} &= \bar{A}\bar{B}C_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + AB C_{in} \\ &= C_{in}(\bar{A}\bar{B} + AB) + AB(\bar{C}_{in} + C_{in}) \\ &= C_{in}(A \oplus B) + AB \end{aligned}$$

But we know that for half adder

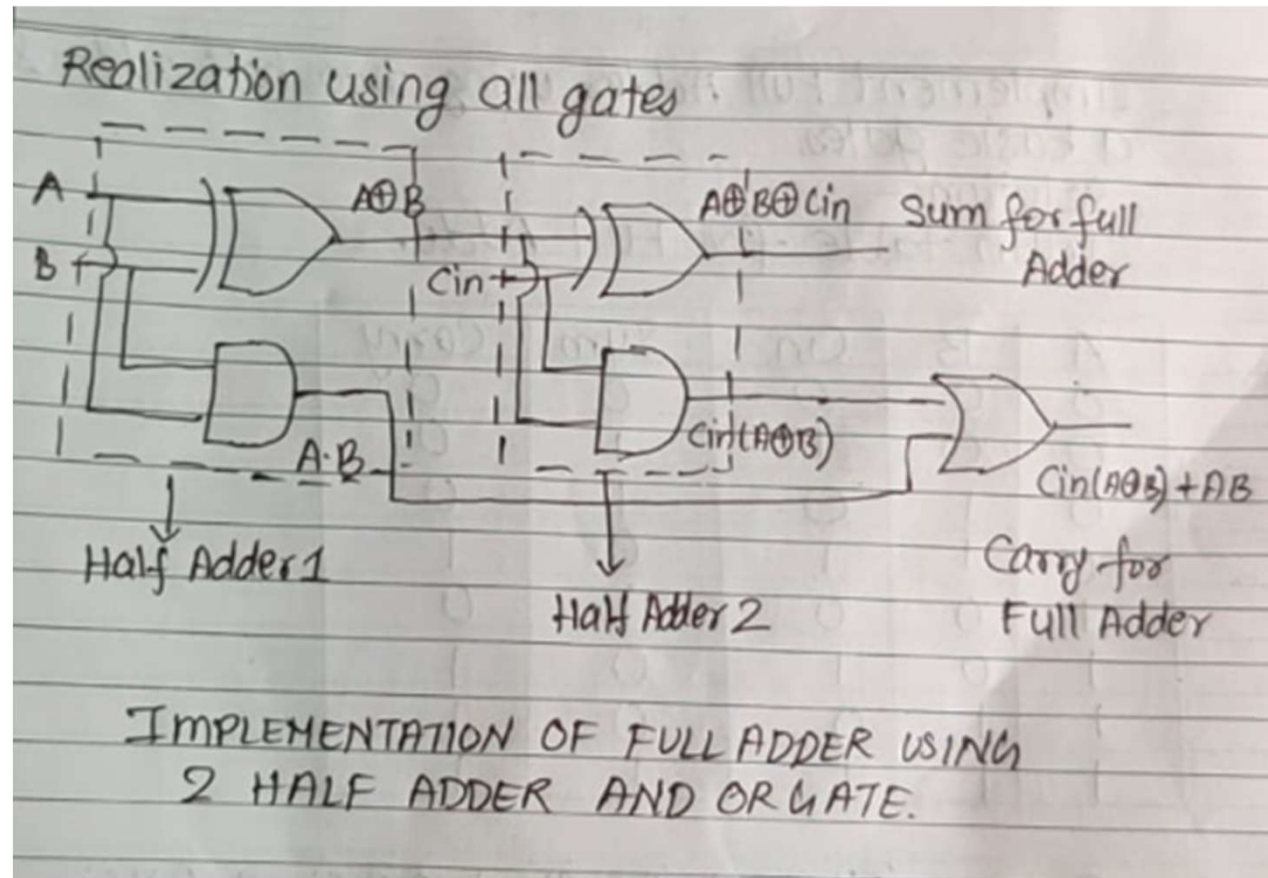
$$\begin{aligned} \text{Sum} &= A \oplus B \\ \text{Carry} &= AB \end{aligned}$$

∴

```

graph LR
    A --> HA1[HA]
    B --> HA1
    HA1 -- Sum (A ⊕ B) --> HA2[HA]
    HA1 -- Carry (AB) --> OR[OR]
    Cin --> HA2
    HA2 -- Sum (A ⊕ B ⊕ C) --> SumOut[Sum]
    HA2 -- Carry ((A ⊕ B)Cin) --> OR
    OR -- Carry --> CarryOut[Carry]
  
```


FULL ADDER USING TWO HALF ADDERS



FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

Inputs			Outputs	
A	B	Bin	Difference ^D	Bout ^B Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

		B Bin			
A		00	01	11	10
0	0	0	1	0	1
1	1	1	0	1	0

Difference $D = \bar{A}\bar{B}Bin + \bar{A}B\bar{B}in + A\bar{B}\bar{B}in + ABBin$

		B Bin			
A		00	01	11	10
0	0	0	1	1	1
1	0	0	0	1	0

Borrow $Bowl = \bar{A}Bin + \bar{A}B + BBin$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

The borrow output for circuit shown, can be given as

$$B_{out} = \bar{A} B_{in} + \bar{A} B + B B_{in}$$

$$= \bar{A} B_{in} (B + \bar{B}) + \bar{A} B + B B_{in} (A + \bar{A})$$

$$= \bar{A} B B_{in} + \bar{A} \bar{B} B_{in} + \bar{A} B + A B B_{in} + \bar{A} B B_{in}$$

$$= \bar{A} B (B_{in} + 1 + B_{in}) + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + B_{in} (\bar{A} \bar{B} + A B)$$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

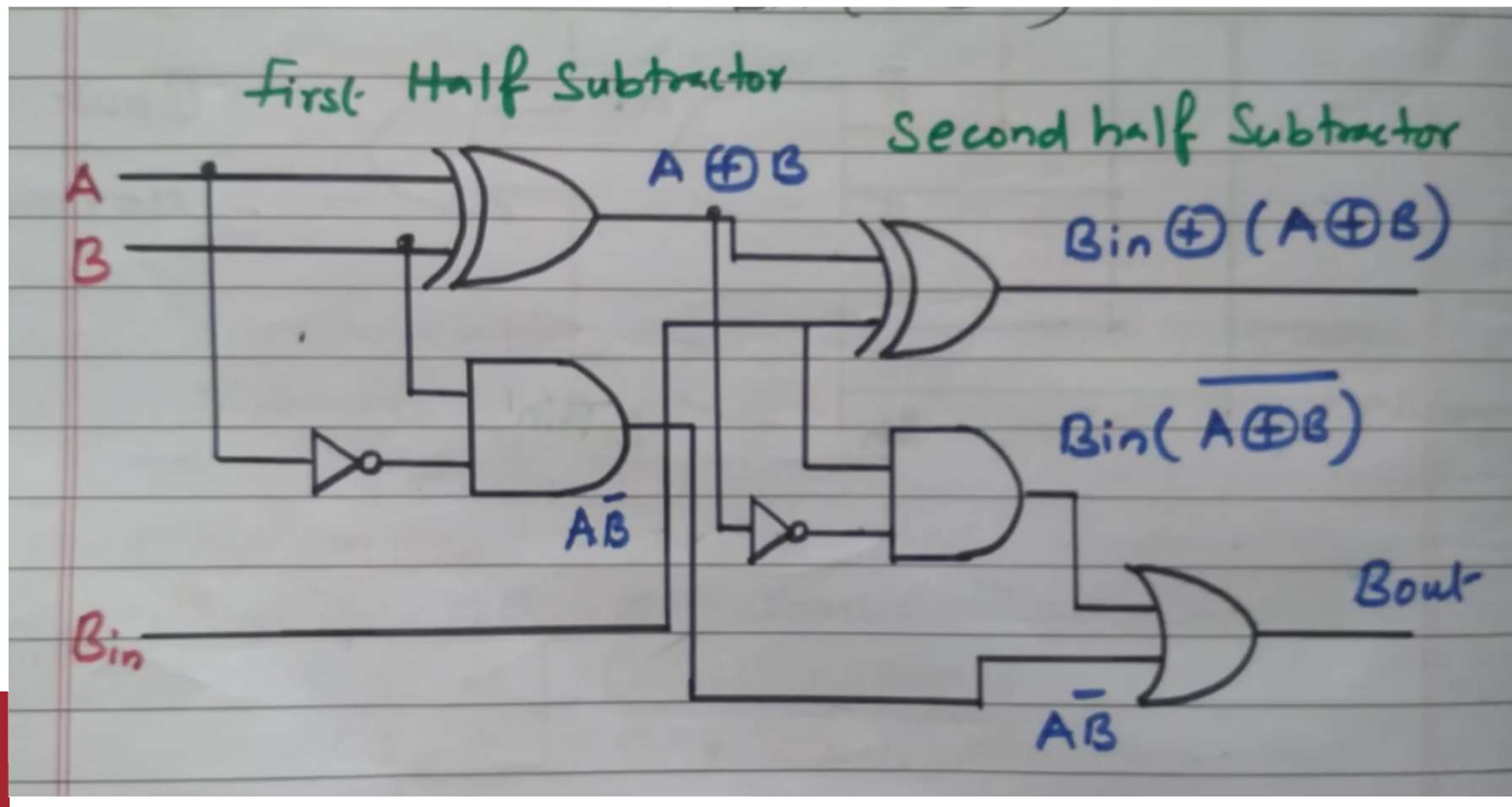
$$\text{Bout} = \overline{A}B + B\overline{A} + (A \oplus B)$$

A full Subtractor can also be implemented with two half Subtractors and one OR gate

$$D = B\overline{A} + (A \oplus B)$$

$$\text{Bout} = \overline{A}B + B\overline{A} + (A \oplus B)$$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR



CONTROLLED ADDER/SUBTRACTOR

- A Controlled Adder Subtractor is essentially the "Swiss Army Knife" of binary arithmetic.
- Instead of having one circuit for addition and a totally separate one for subtraction, a clever trick is used to combine both into a single piece of hardware.
- The circuit uses a Control Signal (usually labeled as M or Mode) to decide which operation to perform

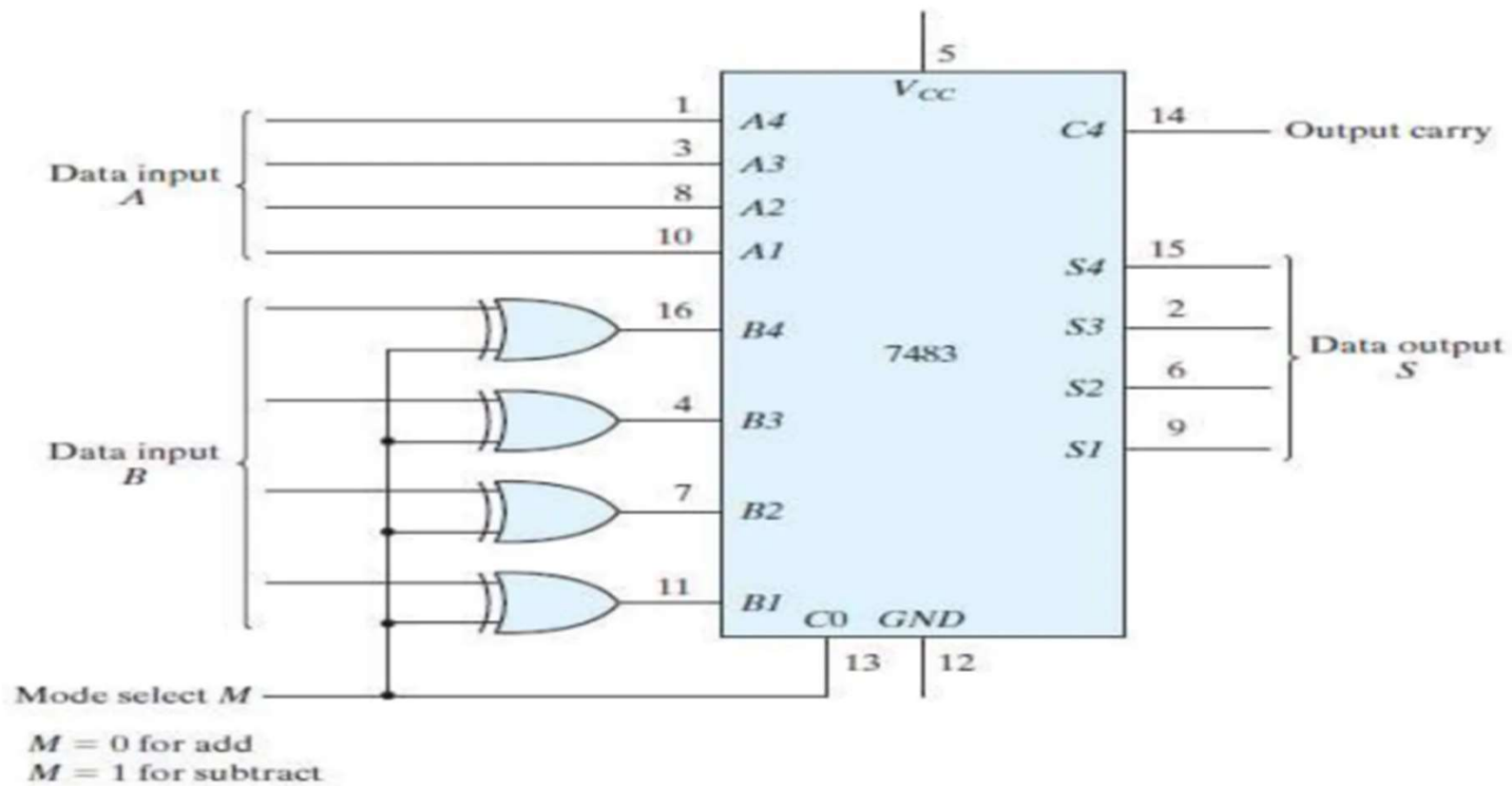
How It Works: The "Magic" of the XOR Gate

- ❑ The heart of this circuit is the XOR gate.
- ❑ In this setup, the XOR gate acts as a conditional inverter.

Remember the XOR rules for an input B and a control signal M:

1. If $M = 0$: $B \oplus 0 = B$ (The input stays the same).
2. If $M = 1$: $B \oplus 1 = \bar{B}$ (The input is inverted/flipped).

IC 7483



CONTROLLED ADDER/SUBTRACTOR (cont.)

1. Addition Mode ($M = 0$)

- When you set the control signal to 0:
- **The XOR Gates:** They pass the B inputs through without changing them ($B \oplus 0 = B$)
- **The Carry-In (C_{in}) :** The M signal is also connected to the very first carry-in of the adder chain.
- So, $C_{in} = 0$.
- **The Result:** The circuit performs a standard binary addition:
- **Sum = A + B + 0**

CONTROLLED ADDER/SUBTRACTOR (cont.)

1. Addition Mode ($M = 0$)

- When you set the control signal to 0:
- **The XOR Gates:** They pass the B inputs through without changing them ($B \oplus 0 = B$)
- **The Carry-In (C_{in}) :** The M signal is also connected to the very first carry-in of the adder chain.
- So, $C_{in} = 0$.
- **The Result:** The circuit performs a standard binary addition:
- **Sum = A + B + 0**

CONTROLLED ADDER/SUBTRACTOR (cont..)

2. Subtraction Mode (M = 1):

When you flip the control signal to 1, the circuit uses 2's Complement logic to subtract:

The XOR Gates: They flip all the bits of B ($B \oplus 1 = \bar{B}$).

This gives us the 1's complement.

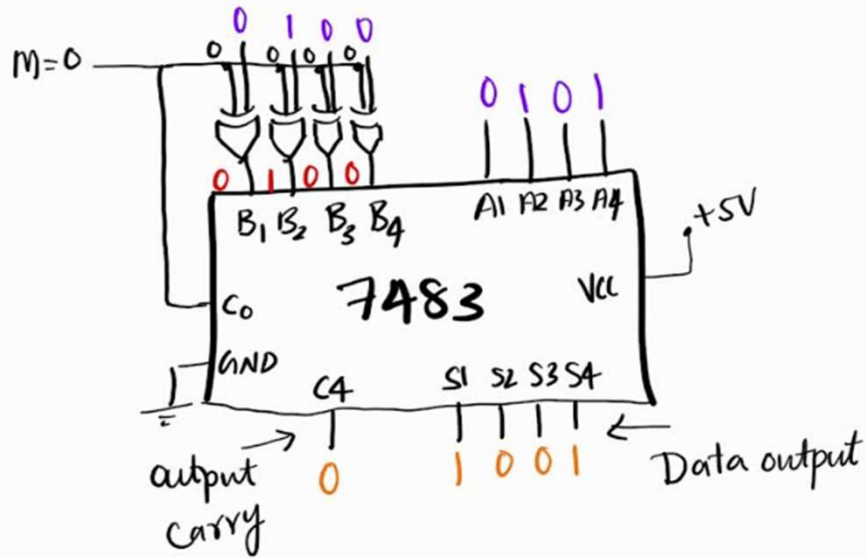
The Carry-In (C_{in}): Since M = 1, the first carry-in is now 1.

The Result: Adding 1 to the 1's complement gives us the **2's complement**.

The circuit performs : **Sum = A + $\bar{B}$ + 1**

(Which mathematically is exactly the same as A - B)

IC 7483 Example for Addition



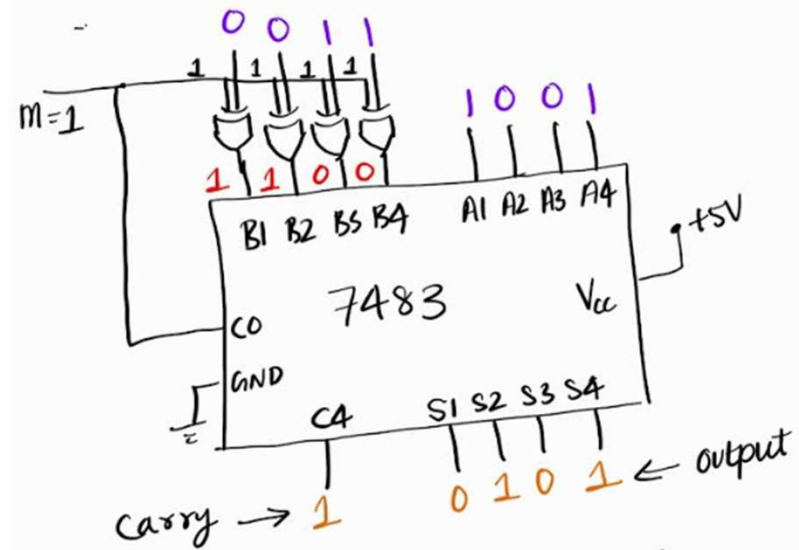
Eg:

$$\begin{array}{r} 5 \\ + 4 \\ \hline 9 \end{array}$$

$$\begin{array}{r} 0101 \\ 0100 \\ \hline (1001)_2 \end{array}$$

$$\begin{array}{r} 0100 \\ 0101 \\ \hline (1001)_2 \end{array}$$

IC 7483 Example for subtraction



Let us consider subtraction of

$$\begin{array}{r} 9 \\ - 3 \\ \hline 6 \end{array}$$

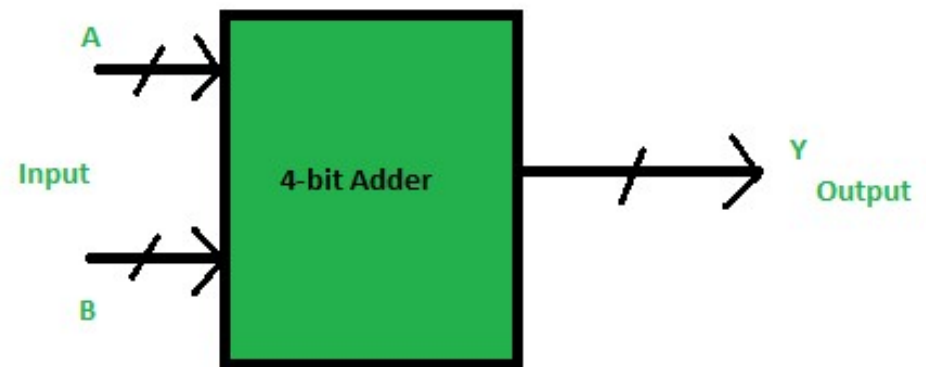
$$\begin{array}{r} 1001 \\ 0011 \\ \hline (0110)_2 \end{array}$$

$$\begin{array}{r} 1001 \\ 1100 \\ \hline 0101 \\ \rightarrow 1 \\ \hline 0110 \end{array}$$

CARRY
WILL BE
ADDED
DUE TO
ONE'S
COMPLEMENT

BCD ADDER

- BCD stands for binary coded decimal.
- It is used to perform the addition of BCD numbers.
- BCD digit can have any of ten possible four-bit representations. Suppose, we have two 4-bit numbers A and B.
- The value of A and B can vary from 0(0000 in binary) to 9(1001 in binary) because we are considering decimal numbers.
- The output will vary from 0 to 18 if we are not considering the carry from the previous sum.
- But if we are considering the carry, then the maximum value of output will be 19 (i.e. $9+9+1 = 19$).
- When we are simply adding A and B, then we get the binary sum. Here, to get the output in BCD form, we will use BCD Adder.



BCD ADDER

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

BCD ADDER

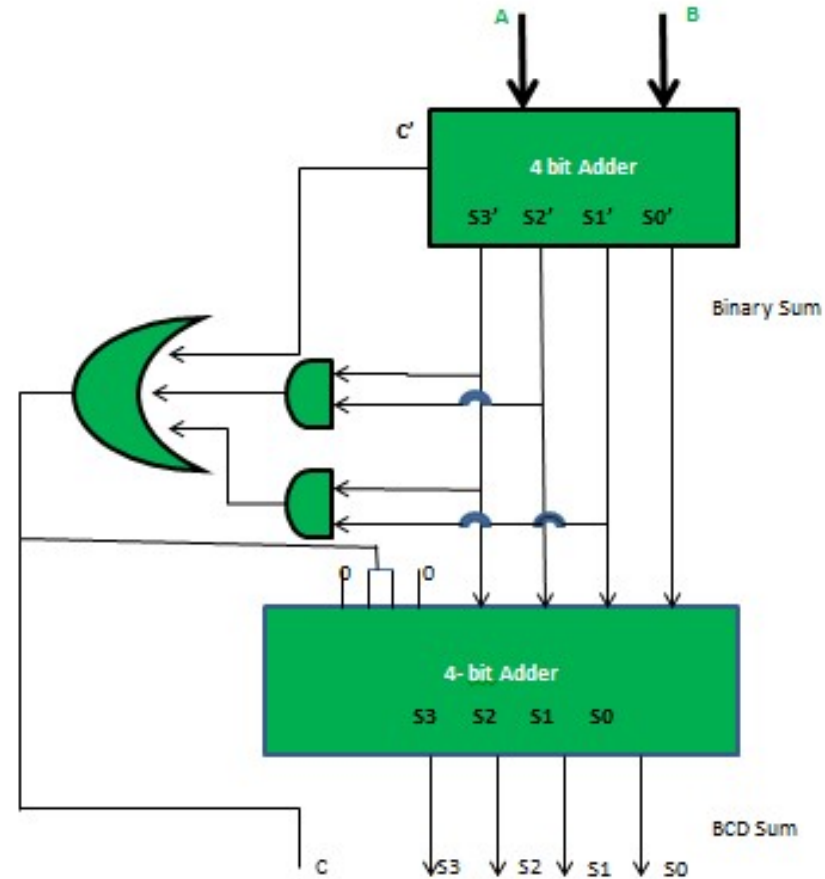
We are adding "0110" (=6) only to the second half of the table. **The conditions are:**

1. If $C' = 1$ (Satisfies 16-19)
2. If $S3'.S2' = 1$ (Satisfies 12-15)
3. If $S3'.S1' = 1$ (Satisfies 10 and 11)

So, our logic is

$$C' + S3'.S2' + S3'.S1' = 1$$

BCD ADDER



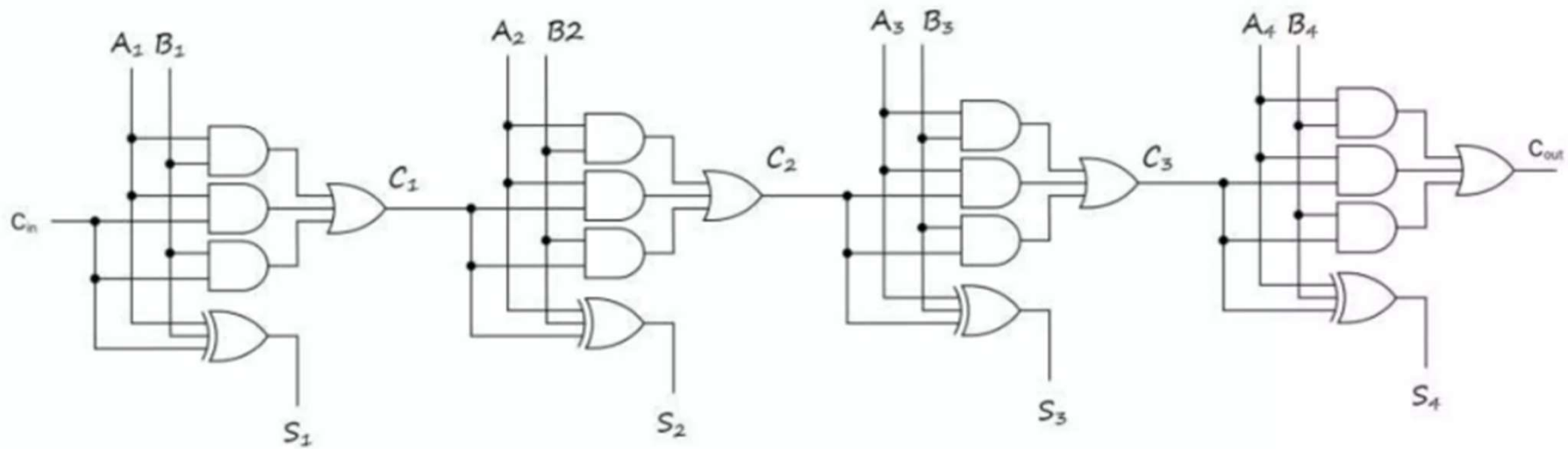
RIPPLE CARRY ADDER

- The ripple carry adder is constructed by cascading full adder blocks in series
- The carryout of one stage is fed directly to the carry-in of the next stage
- This adder is called a ripple carry adder because each carry bit gets rippled into the next stage.
- For an n -bit ripple adder, we require n full adders

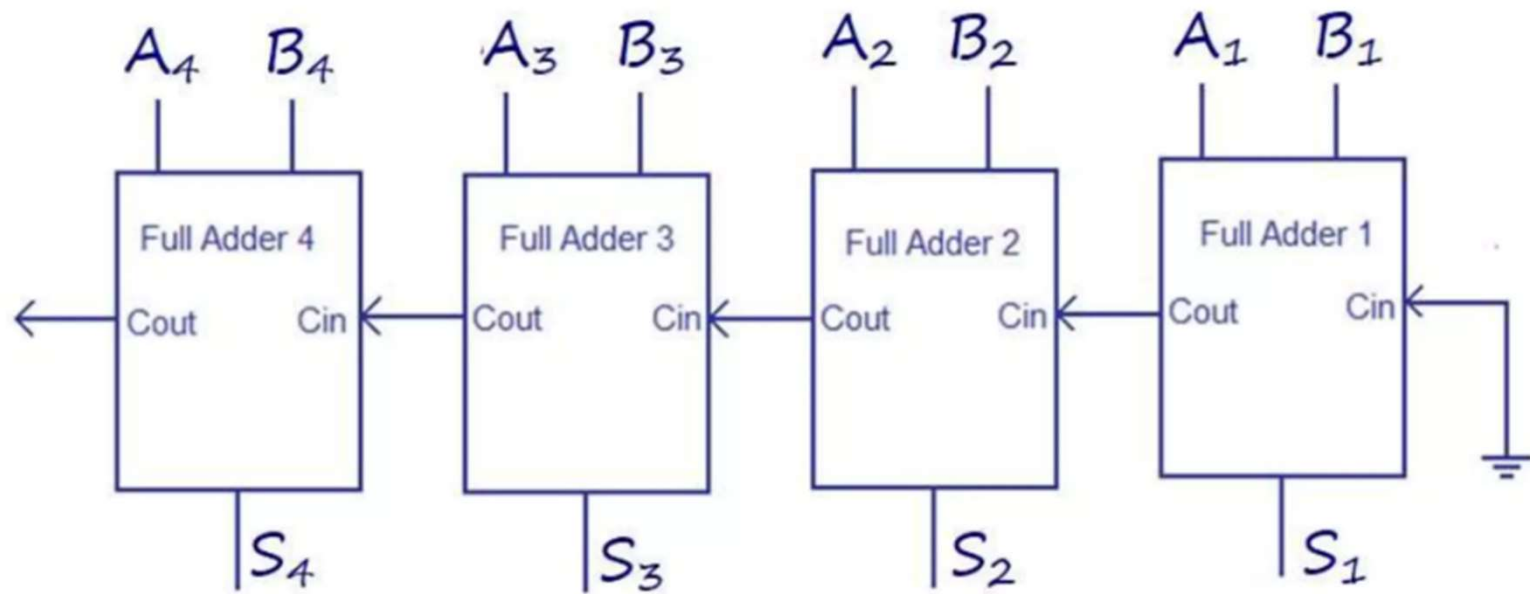
RIPPLE CARRY ADDER

- Propagation delays inside the logic circuitry is the reason behind this
- Propagation delay is time elapsed between the application of an input and occurrence of the corresponding output.
- Consider a NOT gate, The time taken for the NOT gate's output to become "0" after the application of logic "1" to the NOT gate's input is the propagation delay here.
- Similarly the carry propagation delay is the time elapsed between the application of the carry in signal and the occurrence of the carry out (C_{out}) signal.

RIPPLE CARRY ADDER DIAGRAM



RIPPLE CARRY ADDER BLOCK DIAGRAM



RIPPLE CARRY ADDER TRUTH TABLE

[illegible]

PYQs

- State the difference between combinational and sequential circuits. -2023
- Implement Full Adder using Two Half Adders and few gates. -2023
- Design a 4-bit BCD Adder -2025
- Design and Implement 1 bit binary Full Adder using basic gates. -2025
- Design a Full Adder using half adder. -2021,2024
- Implement a full subtractor using two half subtractors. -2024,2023
- Implement a one digit BCD adder with a suitable diagram and equation. -2024
- Implement a Full adder with suitable diagram and truth table. -2024
- Explain logic for implementing BCD adder using 4 Bit adder Device (Binary adder). -2024
- Draw circuit diagram of controlled adder/subtractor using 4 bit adder IC and explain with example. -2024
- Differentiate between combinational and sequential circuit. -2024,2023,2022
- Design half adder using basic gates. -2024
- Design half adder realize using logic gates. -2024
- Design a full adder. Explain the working and application for the same. -2024



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



PYQs

- 2023
 1. Implement a controlled adder/subtractor using 4 bit adder IC and additional gates (5 marks)
 2. Implement a 4 bit adder using full adders (5 marks)
 3. Draw the block diagram of a 4 bit adder/subtractor. Briefly explain how subtraction is done. (5 marks)
 4. Design a 4 bit Ripple carry adder using full or half adder blocks. Label the circuit and explain the working using example $(1001)_2 + (1001)_2$ (10 marks)
 5. Implement 4 bit adder/subtractor using MSI. (Pin numbers of MSI device are not expected). Explain operation with suitable example. (10 marks)